

API basics

What is API..?

API stands for Application Programming Interface.

👉 An API is a bridge that allows two software systems to talk to each other.

An API:

- Accepts a request
- Processes it
- Returns a response

You open Instagram → App asks server:

GET /user/123

Server responds:

```
{  
  "name": "Prajwal",  
  "followers": 1200  
}
```

That communication is done via an API.

🔥 Types of APIs (High Level)

1. Web APIs (REST, GraphQL) – Most common
2. Library APIs – Like `fs.readFile()` in Node
3. OS APIs – Like Windows file system access
4. Third-party APIs – Stripe, Google Maps, etc.

APIs allow:

- Frontend ↔ Backend communication
- Microservices communication

- Third-party integrations
- Mobile app ↔ Server interaction
- AI integrations (very important in future stack)

How web works..?

Modern web works heavily using APIs.

The web works like this:

Browser ↔ Server

That communication happens through:

HTTP requests and responses

And those endpoints on the server are called:

APIs

◆ Traditional Website (Old Style)

Browser requests:

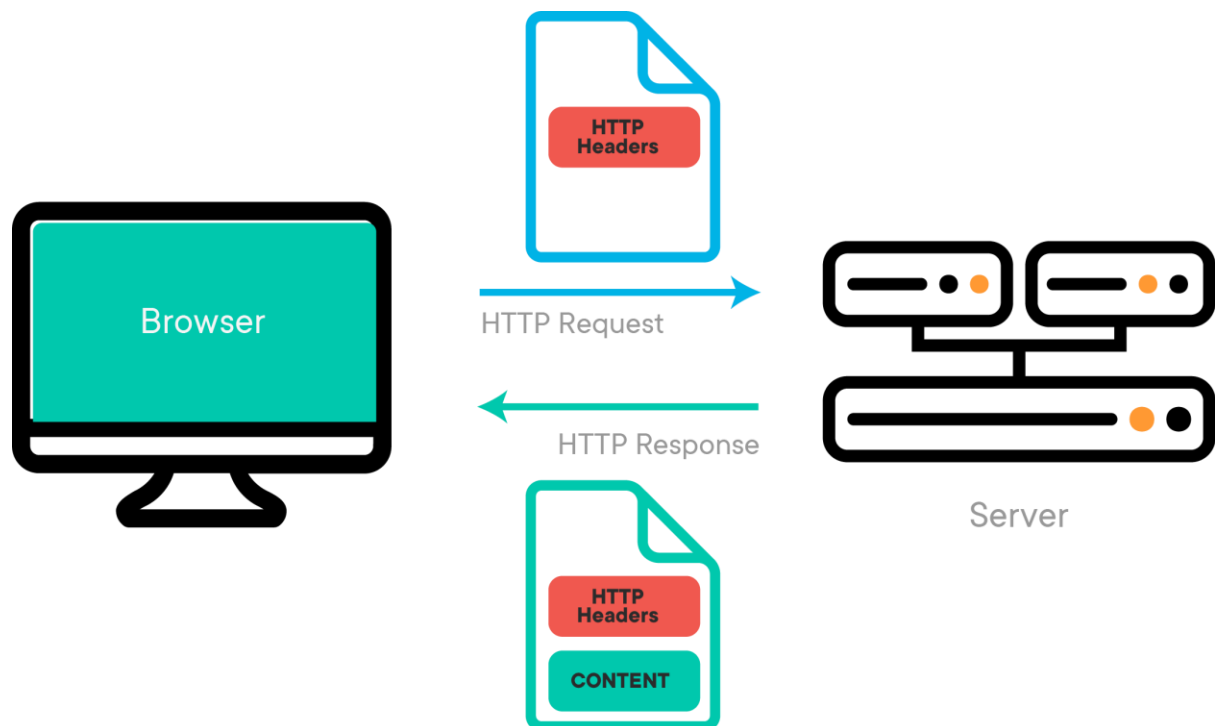
GET /home

Server sends full HTML page.

Browser reloads completely.

No separate API concept visible.

The web works by sending HTTP requests, and in modern applications those requests are made to APIs.



REST Principles

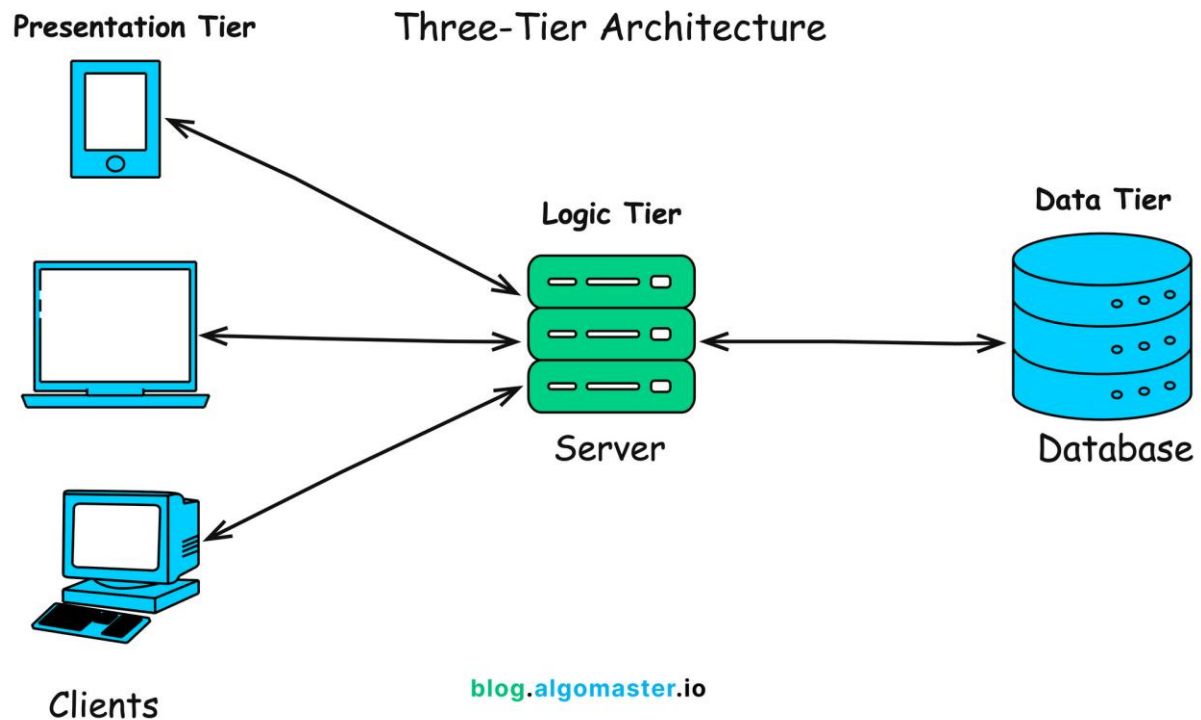
REST = Representational State Transfer

It's a way to design APIs properly.

REST is a set of architectural principles for designing scalable, stateless, resource-based web APIs using HTTP.

Think of REST as rules for building clean, scalable web APIs.

1 Client–Server Separation



- Frontend (React) handles UI
- Backend (Node/Express) handles logic & database
- They communicate via API

👉 They must be independent.

🔑 2 Stateless

Each request must contain all information needed.

Server does NOT remember previous requests.

Example:

GET /profile

Authorization: Bearer token

If you don't send token → server won't know who you are.

👉 Every request is self-contained.

3 Resource-Based URLs

Everything is treated as a resource (noun).

Good REST:

GET /users

GET /users/10

POST /users

Bad REST:

GET /getUsers

POST /createUser

 Use nouns, not verbs.

4 HTTP Methods Matter

Method	Meaning
GET	Read
POST	Create
PUT	Update (full)
PATCH	Update (partial)
DELETE	Remove

5 Uniform Interface

Consistent structure everywhere:

- Same URL pattern
- Same response format (JSON)
- Same error structure

This makes APIs predictable.

6 Cacheable

Responses should tell whether they can be cached.

Example header:

Cache-Control: max-age=3600

Improves performance.

7 Layered System

Client doesn't know if it's talking to:

- Direct server
- Load balancer
- Proxy
- CDN

It just calls the API.

gRPC

What is gRPC?

gRPC = Google Remote Procedure Call

It's a high-performance communication protocol used for service-to-service communication.

Instead of using normal REST (JSON over HTTP),
gRPC uses:

- ⚡ HTTP/2
- 📦 Protocol Buffers (binary format)
- 🔄 Bi-directional streaming

Simple Definition

gRPC lets one server call a function on another server as if it were local.

- 1 Client defines service in .proto file
- 2 Code is auto-generated
- 3 Client calls method
- 4 Data is serialized in binary
- 5 Server processes and responds

gRPC vs REST

Feature	REST	gRPC
Data Format	JSON	Binary (Protobuf)
Speed	Slower	Faster
Browser Friendly	Yes	Not directly
Streaming	Limited	Built-in
Schema	Optional	Required

When to Use gRPC?

- ✓ Microservices
- ✓ Internal service communication
- ✓ High performance systems
- ✓ Real-time streaming
- ✓ Large-scale backend systems

Not ideal for:

- Public browser APIs
- Simple CRUD apps

Inside Netflix-like architecture:

User Service ↔ Payment Service ↔ Recommendation Service

These microservices communicate using gRPC for speed.

Why Big Companies Use It?

- **Google**
- **Netflix**
- **Uber**
- **Square**

Because:

- **Low latency**
- **Efficient**
- **Strong contract-based API**
- **Auto-generated code**

One Line Summary

gRPC is a fast, binary, HTTP/2-based communication protocol mainly used for microservices.

HTTP

HTTP is the language of the web.

Browser ↔ Server communicate using HTTP messages.

1 🔥 HTTP Methods (What Action You Want)

📌 Common Methods

Method	Purpose	Idempotent?
GET	Read data	✅ Yes
POST	Create data	❌ No
PUT	Replace data	✅ Yes
PATCH	Update partial	❌ Usually
DELETE	Remove data	✅ Yes

◆ GET (Fetch Data)

GET /users/10 HTTP/1.1

Host: example.com

✓ Retrieves data

✓ No body usually

✓ Safe & idempotent

If called 10 times → same result.

◆ POST (Create)

POST /users HTTP/1.1

Content-Type: application/json

{

```
"name": "Prajwal"
}
```

- ✓ Creates resource
- ✓ Has body
- ✗ Not idempotent

Calling twice → creates two users.

◆ PUT (Full Update)

```
PUT /users/10
{
  "name": "New Name"
}
```

- ✓ Replaces entire resource
- ✓ Idempotent

Call 10 times → still same final state.

◆ PATCH (Partial Update)

```
PATCH /users/10
{
  "name": "Updated"
}
```

- ✓ Updates specific fields
- ✓ Not always idempotent

◆ DELETE

```
DELETE /users/10
```

- ✓ Deletes resource
- ✓ Idempotent

Deleting twice → still deleted.

2 🚦 HTTP Status Codes (Server Response Meaning)

Status codes are 3-digit numbers.

◆ 1xx — Informational

Rarely used in apps.

◆ 2xx — Success

Code	Meaning
200	OK
201	Created
204	No Content

◆ 3xx — Redirection

Code	Meaning
301	Permanent redirect
302	Temporary redirect
304	Not Modified (cache)

◆ 4xx — Client Errors

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error

Client did something wrong.

◆ 5xx — Server Errors

Code	Meaning
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Server failed.

3 📦 HTTP Message Format

HTTP has:

- Request format
- Response format

◆ HTTP Request Format

METHOD /path HTTP/version
Headers

Body (optional)

Example:

POST /users HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer token

```
{  
  "name": "Prajwal"  
}
```

◆ HTTP Response Format

HTTP/version StatusCode StatusMessage
Headers

Body

Example:

HTTP/1.1 201 Created

Content-Type: application/json

```
{  
  "id": 10,  
  "name": "Prajwal"  
}
```

Questions

 **1** What happens when you type a URL in the browser?

 Interview-Ready Answer:

When I enter a URL:

1. DNS resolves domain to IP address
2. Browser establishes TCP connection (3-way handshake)
3. If HTTPS → TLS handshake happens
4. Browser sends HTTP request
5. Server processes request
6. Server sends HTTP response
7. Browser parses HTML, loads CSS/JS, renders page

This is the complete web request lifecycle.



2 What is HTTP?



Answer:

HTTP (HyperText Transfer Protocol) is a stateless application-layer protocol used for communication between client and server using a request–response model.

It runs over TCP and defines how messages are structured and transmitted.



3 What is REST?



Answer:

REST is an architectural style for designing networked APIs.

It is:

- **Stateless**
- **Resource-based**
- **Uses standard HTTP methods**
- **Follows uniform interface**

REST APIs operate on resources using URLs and HTTP verbs.



4 What is Statelessness?



Answer:

Statelessness means the server does not store client context between requests.

Each request must contain all necessary information (like authentication token).

This improves scalability because any server can handle any request.

 5 What is Idempotency?

 **Answer:**

An HTTP method is idempotent if making the same request multiple times results in the same final state.

GET, PUT, DELETE are idempotent.
POST is not.

 6 PUT vs PATCH?

 **Answer:**

PUT replaces the entire resource.
PATCH updates only specific fields.

PUT is idempotent.
PATCH may or may not be idempotent.

 7 401 vs 403?

 **Answer:**

401 Unauthorized → Authentication is required or failed.

403 Forbidden → User is authenticated but does not have permission.

 8 HTTP vs HTTPS?

 **Answer:**

HTTPS is HTTP over TLS/SSL encryption.

It ensures:

- Data encryption
- Data integrity
- Server authentication

HTTP sends data in plain text.

  REST vs gRPC?

 Answer:

REST:

- Uses HTTP/1.1
- JSON format
- Easy for browsers
- Human readable

gRPC:

- Uses HTTP/2
- Binary (Protocol Buffers)
- Faster
- Better for microservices

Use REST for public APIs.

Use gRPC for internal service communication.

  What is CORS?

 Answer:

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts requests from different origins.

Server must explicitly allow cross-origin requests using headers.

   How do you design a REST API?

 Answer (Short & Strong):

- Use nouns for resources (/users)
- Use proper HTTP methods
- Keep it stateless

- Use proper status codes
- Implement pagination & filtering
- Secure with authentication
- Version the API (/api/v1)

 **1 2** What is HTTP/2 improvement over HTTP/1.1?

 Answer:

HTTP/2 introduced:

- Multiplexing (multiple requests in one connection)
- Header compression
- Server push
- Binary framing

It improves performance and reduces latency.